

Python for Machine Learning

The Essential Tools

Meyssem Bouzaïen

Academic Year: 2025 - 2026

Course Outline

- 1 Introduction and Environment
- 2 NumPy - Numerical Computation
- 3 Pandas - Data Manipulation
- 4 Understanding Correlation
- 5 Matplotlib - Visualization
- 6 Scikit-learn - First ML Model
- 7 Workflow and Best Practices

Course Objectives

What you will learn

- Master NumPy, Pandas, Matplotlib, and Scikit-learn
- Manipulate and prepare real-world data
- Visualize your results
- Build your first ML model

Working Environment: You Have a Choice!

Available options:

- Google Colab (recommended)
- Jupyter Notebook
- Anaconda + Jupyter
- VS Code
- PyCharm

Use whichever environment you prefer!

Why Google Colab?

- Free
- No installation
- GPU available
- Easy to share

The 4 Essential Libraries

NumPy

Numerical computation

Pandas

Data manipulation

Matplotlib

Visualization

Scikit-learn

ML algorithms

NumPy: What Exactly Is It?

Definition

NumPy (Numerical Python) is the fundamental library for scientific computing in Python. It provides high-performance data structures for working with multidimensional arrays.

Characteristics:

- N-dimensional arrays
- Vectorized operations
- Optimized in C
- Foundation for Pandas, Scikit-learn

Data types:

- int32, int64
- float32, float64
- bool, complex
- Homogeneous within an array

NumPy: Why?

Python lists:

```
1 my_list = [1, 2, 3, 4]
2 result = []
3 for x in my_list:
4     result.append(x * 2)
5 # Slow!
```

NumPy arrays:

```
1 import numpy as np
2 array = np.array([1,2,3,4])
3 result = array * 2
4 # Fast!
```

NumPy Advantages

- 50-100x faster than lists
- Simple syntax
- Less memory usage

1D vs 2D vs 3D Arrays - Visualization

1D Array

5	8	3	9
---	---	---	---

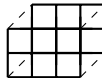
shape: (4,)

2D Array (Matrix)

11	12	13	14	15
6	7	8	9	10
1	2	3	4	5

shape: (3, 5)

3D Array



shape: (3, 3, 2)

Creating Arrays

```
import numpy as np

# From a list
a = np.array([1, 2, 3, 4, 5])
print(a)  # [1 2 3 4 5]

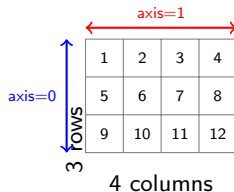
# 2D matrix
m = np.array([[1, 2, 3],
              [4, 5, 6]])
print(m.shape)  # (2, 3)

# Special arrays
zeros = np.zeros((3, 4))      # Matrix of zeros
ones = np.ones((2, 3))       # Matrix of ones
range_arr = np.arange(0, 10, 2)  # [0 2 4 6 8]
linear = np.linspace(0, 1, 5)  # 5 values [0, 0.25, 0.5, 0.75, 1]
random = np.random.rand(3, 3)  # Random values in [0, 1)
```

Dimensions and Shapes

```
data = np.array([[1, 2, 3, 4],
                 [5, 6, 7, 8],
                 [9, 10, 11, 12]])

print(data.shape)  # (3, 4)
print(data.ndim)   # 2
print(data.size)   # 12
print(data.dtype)  # int64
```



Important

axis=0: operations across rows (vertical)

axis=1: operations across columns (horizontal)

How NumPy operates on arrays of different sizes

Example 1: Array + Scalar

$$\begin{bmatrix} 1 & 2 & 3 & 4 \end{bmatrix} + \boxed{5} = \begin{bmatrix} 6 & 7 & 8 & 9 \end{bmatrix}$$

Example 2: Matrix + Vector

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix} + \begin{bmatrix} 10 & 20 & 30 \end{bmatrix} = \begin{bmatrix} 11 & 22 & 33 \\ 14 & 25 & 36 \\ 17 & 28 & 39 \end{bmatrix}$$

Mathematical Operations

Basic operations:

```
1 a = np.array([1,2,3,4])
2 b = np.array([10,20,30,40])
3
4 a + b # [11 22 33 44]
5 a * b # [10 40 90 160]
6 a ** 2 # [1 4 9 16]
7 a / 2 # [0.5 1 1.5 2]
```

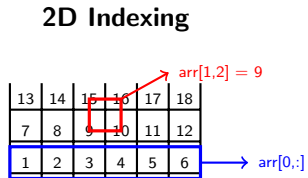
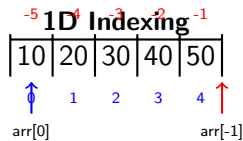
Useful functions:

```
1 data = np.array([1,2,3,4,5])
2
3 np.mean(data) # 3.0
4 np.sum(data) # 15
5 np.min(data) # 1
6 np.max(data) # 5
7 np.std(data) # Standard deviation
8 np.sqrt(data) # Square roots
9 np.median(data) # Median
```

Important

All operations are performed element-wise!

Indexing - Visualization



Indexing and Slicing

```
# 1D Array
arr = np.array([10, 20, 30, 40, 50])
arr[0]      # 10 - first element
arr[-1]     # 50 - last element
arr[1:4]    # [20 30 40] - slicing

# 2D Array
mat = np.array([[1, 2, 3],
                [4, 5, 6],
                [7, 8, 9]])

mat[0, 0]   # 1 - first row, first column
mat[1, :]   # [4 5 6] - the whole row 1
mat[:, 2]   # [3 6 9] - the whole column 2
mat[0:2, 1:3] # [[2 3]
               #  [5 6]] - sub-matrix
```

NumPy Practice Exercise

Your turn!

- 1 Create an array of 20 random numbers between 0 and 100
- 2 Compute the mean and standard deviation
- 3 Find the values greater than 50
- 4 Create a 5x4 matrix of random numbers
- 5 Compute the sum of each column

```
# Hint to get started
import numpy as np

# Generate random numbers
data = np.random.randint(0, 100, 20)
```

Pandas: What Is It?

DataFrame = an Excel spreadsheet in Python

- Tabular data structure
- Named columns
- Indexed rows
- Mixed data types

Name	Age	City	Score
Alice	25	Paris	85
Bob	30	Lyon	92
Charlie	22	Marseille	78

This is how we work with data in ML!

Creating a DataFrame (1/2)

```
import pandas as pd

# Method 1: From a dictionary
data = {
    'Name': ['Alice', 'Bob', 'Charlie'],
    'Age': [25, 30, 22],
    'City': ['Paris', 'Lyon', 'Marseille'],
    'Score': [85, 92, 78]
}
df = pd.DataFrame(data)
print(df)
```

	Name	Age	City	Score
0	Alice	25	Paris	85
1	Bob	30	Lyon	92
2	Charlie	22	Marseille	78

Creating a DataFrame (2/2)

Loading from a CSV file:

```
import pandas as pd

# From a CSV file
df = pd.read_csv('data.csv')

# From an Excel file
df = pd.read_excel('data.xlsx')

# From a URL
url = 'https://example.com/data.csv'
df = pd.read_csv(url)
```

First things to check:

```
df.head()      # first 5 rows
df.tail()      # last 5 rows
df.shape       # (rows, columns)
df.info()      # column info
df.describe()  # statistics
```

Exploring the Data

```
# Load the Iris dataset
df = pd.read_csv('iris.csv')

# Dimensions
print(df.shape)  # (150, 5)

# Column names
print(df.columns)

# Data types
print(df.dtypes)

# Descriptive statistics
print(df.describe())

# Count unique values
print(df['species'].value_counts())

# Check for missing values
print(df.isnull().sum())
```

Selecting Data (1/2)

```
# Select a column
ages = df['Age']
# or
ages = df.Age

# Select several columns
subset = df[['Name', 'Score']]

# Filter rows
successful_students = df[df['Score'] > 80]

# Multiple conditions
filtered = df[(df['Age'] > 20) & (df['Score'] > 85)]
```

Selecting Data (2/2)

```
# Select by position (iloc)
df.iloc[0]          # first row
df.iloc[0:3]        # first 3 rows
df.iloc[0:3, 0:2]   # 3 rows, 2 columns

# Select by label (loc)
df.loc[0, 'Name']    # specific value
df.loc[0:2, ['Name', 'Age']] # mixed selection
```

iloc vs loc

- **iloc**: uses positions (0, 1, 2...)
- **loc**: uses labels/names

Operations on Data

```
# Add a new column
df['Double_Score'] = df['Score'] * 2

# Modify a column
df['Age'] = df['Age'] + 1

# Drop a column
df = df.drop('Double_Score', axis=1)

# Drop a row
df = df.drop(0, axis=0)

# Sort the data
df_sorted = df.sort_values('Score', ascending=False)

# Group and aggregate
averages = df.groupby('City')['Score'].mean()
```

Handling Missing Values

```
1 # Check missing values
2 print(df.isnull().sum())
3
4 # Drop rows with missing values
5 df_clean = df.dropna()
6
7 # Drop columns with missing values
8 df_clean = df.dropna(axis=1)
9
10 # Fill with a value
11 df_filled = df.fillna(0)
12
13 # Fill with the mean
14 df['Age'] = df['Age'].fillna(df['Age'].mean())
15
16 # Fill with the previous value
17 df_filled = df.fillna(method='ffill')
```

Iris Dataset

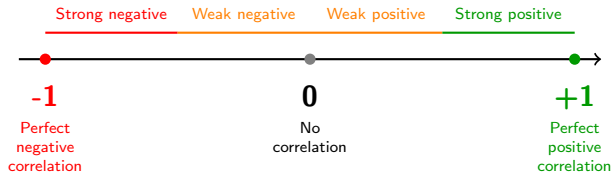
- 1 Load iris.csv
- 2 Display the first 10 rows
- 3 How many rows and columns are there?
- 4 Which species are present?
- 5 Compute the mean of each numeric column
- 6 Filter flowers with sepal_length ≥ 6
- 7 Create: `sepal_area = sepal_length * sepal_width`

```
import pandas as pd
# Your code here...
```


What is Correlation? (1/3)

Definition

Correlation measures the **linear relationship** between two quantitative variables. It ranges between **-1** and **+1**.



What is Correlation? (2/3)

Interpreting the values:

Value	Strength	Meaning
+0.90 to +1.00	Very strong +	Both increase together
+0.70 to +0.89	Strong +	Clear relationship
+0.40 to +0.69	Moderate +	Visible trend
+0.10 to +0.39	Weak +	Slight relationship
-0.09 to +0.09	Negligible	No relationship
-0.10 to -0.39	Weak -	Slight inverse relationship
-0.40 to -0.69	Moderate -	Inverse trend
-0.70 to -0.89	Strong -	Clear inverse relationship
-0.90 to -1.00	Very strong -	One rises, the other falls

What is Correlation? (3/3)

Visual examples:



Important

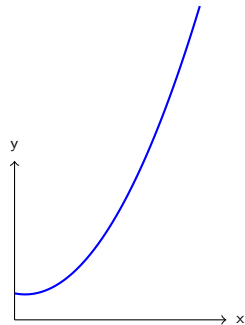
Correlation does NOT mean causation!

Two variables can be correlated without one causing the other.

Why Visualize?

A picture is worth 1000 words

- Understand the data
- Detect patterns
- Identify outliers
- Communicate results
- Verify hypotheses



Visible trend!

In ML

Visualization is essential!

Your First Plot

```
import matplotlib.pyplot as plt

# Data
x = [1, 2, 3, 4, 5]
y = [2, 4, 6, 8, 10]

# Create the plot
plt.plot(x, y)

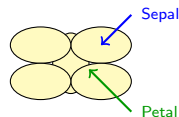
# Customize
plt.xlabel('X axis')
plt.ylabel('Y axis')
plt.title('My first plot')

# Show
plt.show()
```

Simple and effective!

4 Features:

- sepal_length: Sepal length
- sepal_width: Sepal width
- petal_length: Petal length
- petal_width: Petal width



Iris flower

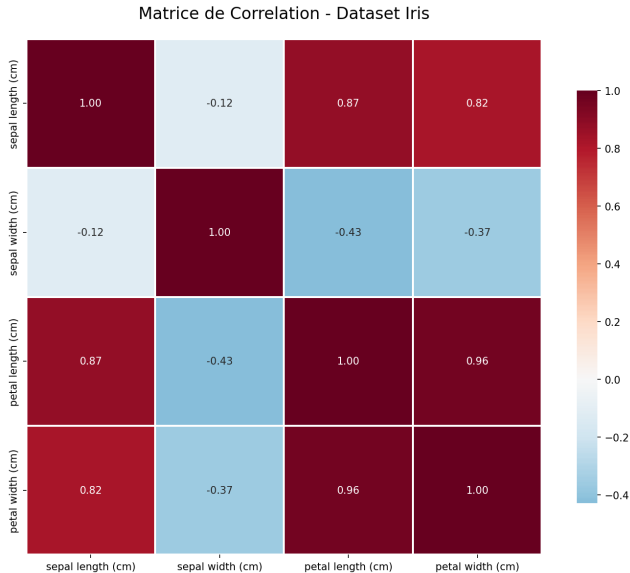
3 Species:

- Setosa
- Versicolor
- Virginica

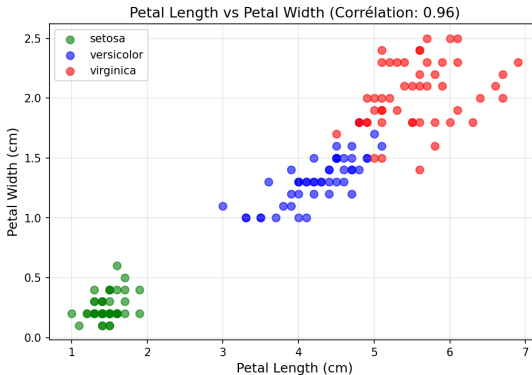
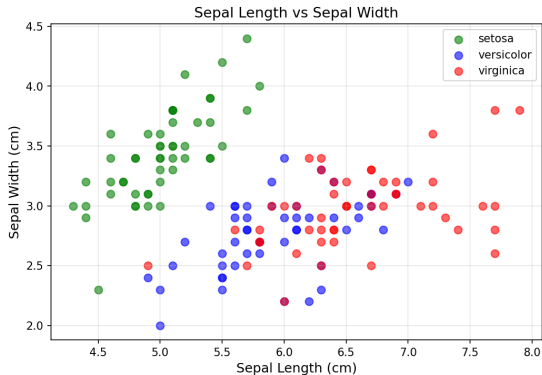
Goal

Predict the species from the 4 measurements

Iris - Correlation Matrix



Iris - Visual Relationships

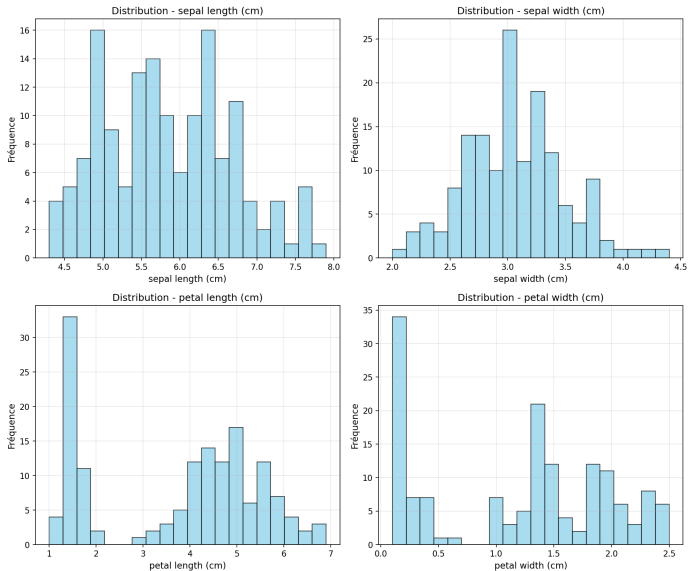


What Do We See?

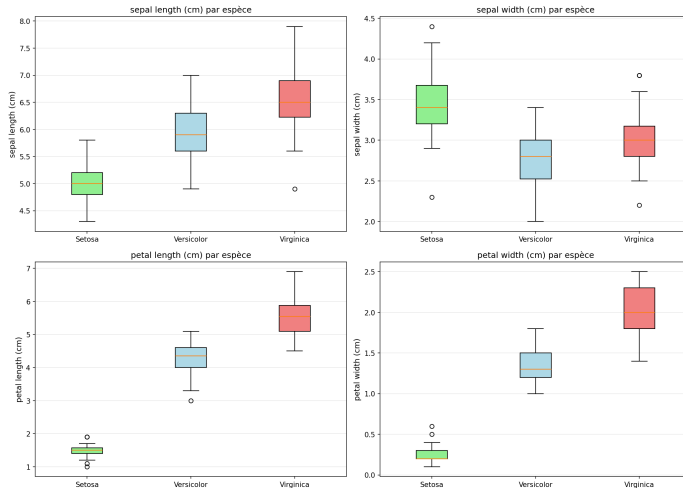
Left: weak correlation (-0.12) - scattered points

Right: very strong correlation ($+0.96$) - clear alignment!

Iris - Histograms by Feature



Iris - Boxplots by Species



Observations

The species are well separated on `petal_length` and `petal_width`!

Chart Types - Scatter Plot

Code:

```
1 import matplotlib.pyplot as plt
2
3 x = [1, 2, 3, 8]
4 y = [1, 4, 5, 3]
5
6 plt.scatter(x, y)
7 plt.xlabel('X')
8 plt.ylabel('Y')
9 plt.title('Scatter plot')
10 plt.show()
```

Result:

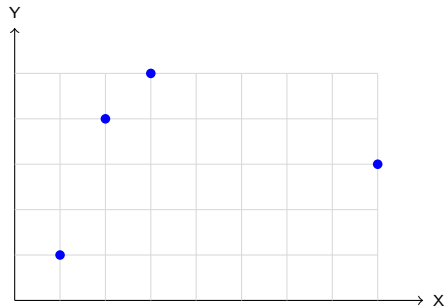


Chart Types - Line Plot

Code:

```
1 import matplotlib.pyplot as plt
2
3 x = [1, 2, 3, 8]
4 y = [0, 4, 5, 3]
5
6 plt.plot(x, y)
7 plt.xlabel('X')
8 plt.ylabel('Y')
9 plt.title('Line plot')
10 plt.show()
```

Result:

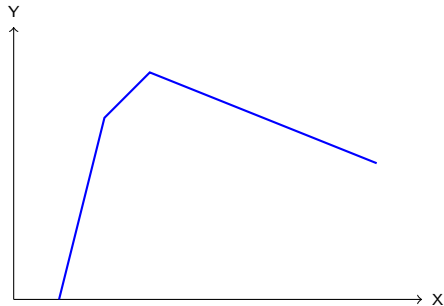


Chart Types - Histogram

Code:

```
1 import matplotlib.pyplot as plt
2 import numpy as np
3
4 data = np.random.randn(1000)
5
6 plt.hist(data, bins=20,
7         color='blue',
8         alpha=0.7)
9 plt.xlabel('Value')
10 plt.ylabel('Frequency')
11 plt.title('Histogram')
12 plt.show()
```

Result:

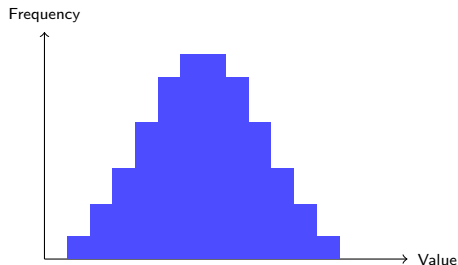
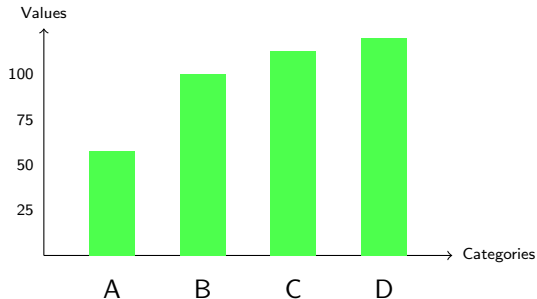


Chart Types - Bar Chart

Code:

```
1 import matplotlib.pyplot as plt
2
3 categories = ['A', 'B', 'C', 'D']
4 values = [23, 45, 56, 78]
5
6 plt.bar(categories, values,
7         color='green')
8 plt.xlabel('Categories')
9 plt.ylabel('Values')
10 plt.title('Bar chart')
11 plt.show()
```

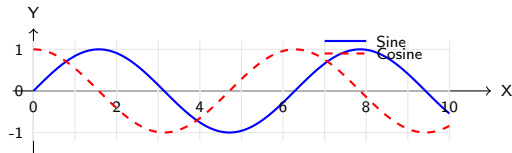
Result:



Multiple Curves - Sine/Cosine Example

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 x = np.linspace(0, 10, 100)
5 y1 = np.sin(x)
6 y2 = np.cos(x)
7
8 plt.plot(x, y1, label='Sine',
9         color='blue', linewidth=2)
10 plt.plot(x, y2, label='Cosine',
11        color='red', linestyle='--',
12        linewidth=2)
13
14 plt.xlabel('X')
15 plt.ylabel('Y')
16 plt.title('Trig Functions')
17 plt.legend()
18 plt.grid(True)
19 plt.show()
```

Result:



Customization

```
# Colors
plt.plot(x, y, color='red') # or 'r', '#FF0000'

# Line styles
plt.plot(x, y, linestyle='--') # '--', '-.', ':'

# Markers
plt.scatter(x, y, marker='o') # 'o', 's', '^', '*'

# Size
plt.plot(x, y, linewidth=3)
plt.scatter(x, y, s=100)

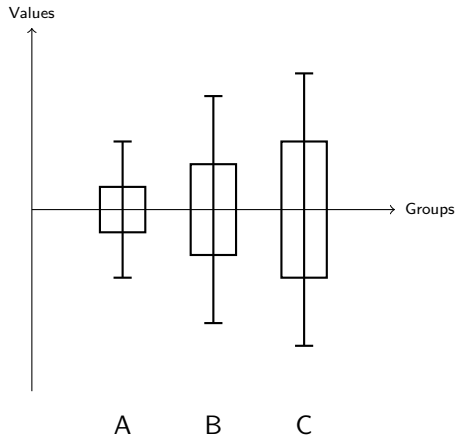
# Transparency
plt.scatter(x, y, alpha=0.5) # 0 to 1
```


Chart Types - Box Plot

Code:

```
1 import matplotlib.pyplot as plt
2 import numpy as np
3
4 data = [np.random.normal(0, std,
5                        100) for std in
6         range(1, 4)]
7
8 plt.boxplot(data,
9             labels=['A', 'B', 'C'])
10 plt.xlabel('Groups')
11 plt.ylabel('Values')
12 plt.title('Boxplot')
13 plt.show()
```

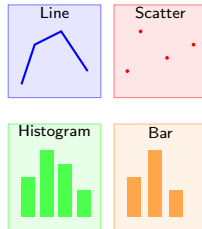
Result:



Subplots - Example

```
fig, axes = plt.subplots(2, 2,  
                          figsize=(10, 8))  
  
# Plot 1  
axes[0, 0].plot(x, y1)  
axes[0, 0].set_title('Line')  
  
# Plot 2  
axes[0, 1].scatter(x, y2)  
axes[0, 1].set_title('Scatter')  
  
# Plot 3  
axes[1, 0].hist(data, bins=20)  
axes[1, 0].set_title('Histogram')  
  
# Plot 4  
axes[1, 1].bar(cat, val)  
axes[1, 1].set_title('Bar')  
  
plt.tight_layout()  
plt.show()
```

Result:



Visualizing with Pandas

```
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv('iris.csv')

# Histogram of a column
df['sepal_length'].hist(bins=20)
plt.show()

# Scatter plot
df.plot.scatter(x='sepal_length', y='sepal_width')
plt.show()

# Box plot by group
df.boxplot(column='petal_length', by='species')
plt.show()
```

Matplotlib Practice Exercise

Visualize Iris

- 1 Histogram for sepal_length
- 2 Scatter plot: sepal_length vs sepal_width
- 3 Color by species
- 4 4 subplots: one histogram per feature
- 5 Customize with titles, labels, colors

```
import pandas as pd
import matplotlib.pyplot as plt

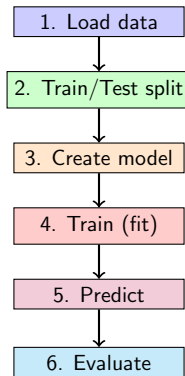
df = pd.read_csv('iris.csv')
# Your code here...
```

Everything for ML

- Classification algorithms
- Regression algorithms
- Clustering algorithms
- Data preprocessing
- Model evaluation
- Feature selection

A simple and consistent interface!

The Scikit-learn Workflow



Iris Classification (1/3)

```
# 1. Imports
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

# 2. Load the data
iris = load_iris()
X = iris.data # Features (4 columns)
y = iris.target # Labels (species: 0, 1, 2)

print("X shape:", X.shape) # (150, 4)
print("y shape:", y.shape) # (150,)
```

Iris Classification (2/3)

```
# 3. Split into train/test (80/20)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

print("Train:", X_train.shape)  # (120, 4)
print("Test:", X_test.shape)    # (30, 4)

# 4. Create the model
model = KNeighborsClassifier(n_neighbors=3)

# 5. Train
model.fit(X_train, y_train)
print("Model trained!")
```


Iris Classification (3/3)

```
# 6. Predictions
y_pred = model.predict(X_test)

# 7. Evaluate
accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy: {accuracy * 100:.2f}%")

# Look at a few predictions
print("\nExamples:")
for i in range(5):
    print(f"Actual: {y_test[i]}, Predicted: {y_pred[i]}")
```

Result

Accuracy: 96-100% on Iris!

Standard Sklearn Structure

```
from sklearn.whatever import MyModel

# 1. Create the model
model = MyModel(parameters)

# 2. Train
model.fit(X_train, y_train)

# 3. Predict
predictions = model.predict(X_test)

# 4. Evaluate
from sklearn.metrics import accuracy_score
score = accuracy_score(y_test, predictions)
```

Important

The same structure applies to ALL sklearn algorithms!

Preprocessing

```
from sklearn.preprocessing import StandardScaler

# Normalize (mean=0, std=1)
scaler = StandardScaler()

# Fit on train, transform train and test
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Use the normalized data
model.fit(X_train_scaled, y_train)
predictions = model.predict(X_test_scaled)
```

Why?

Some algorithms are sensitive to scale!

Your First Model

- 1 Load the Iris dataset
- 2 Split into train (70%) and test (30%)
- 3 Train a KNN with $k=5$
- 4 Make predictions
- 5 Compute and display accuracy
- 6 Show a few predictions vs actual values

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score
```

① Understand the problem

- Classification? Regression? Clustering?

② Explore the data

- `df.info()`, `df.describe()`, visualizations

③ Prepare the data

- Missing values, normalization

④ Train a model

- Start simple (KNN, trees...)

⑤ Evaluate and improve

- Metrics, cross-validation

Complete Template (1/2)

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

# 1. Load
df = pd.read_csv('data.csv')

# 2. Explore
print(df.head())
print(df.info())
df.hist()
plt.show()
```

Complete Template (2/2)

```
# 3. Prepare
X = df.drop('target', axis=1).values
y = df['target'].values

# 4. Split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

# 5. Normalize
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# 6. Train
model = KNeighborsClassifier(n_neighbors=5)
model.fit(X_train, y_train)

# 7. Evaluate
y_pred = model.predict(X_test)
print(f"Accuracy: {accuracy_score(y_test, y_pred):.2f}")
```

Documentation

- NumPy: numpy.org/doc
- Pandas: pandas.pydata.org/docs
- Matplotlib: matplotlib.org
- Scikit-learn: scikit-learn.org

Practice

- Kaggle: datasets and tutorials
- Google Colab: free notebooks
- UCI ML Repository: datasets

Recap

You now know

- **NumPy**: fast numerical computation
- **Pandas**: data manipulation
- **Correlation**: interpreting relationships
- **Matplotlib**: visualization
- **Scikit-learn**: ML models

Next Step

Classification algorithms!